

Rapport 1

Systèmes d'Exploitation – NACHOS

Devoir 1 : Entrées/Sorties dans NACHOS

Mervyn Samsan - Robert Dylan - Negre Damien

October 5, 2025

I. Bilan

- Pour ce premier rendu, nous avons réussi à aller jusqu'au bout du sujet . Aucune question n'a été laissée de côté.
- Tous les appels systèmes demandés de lecture et d'écriture de chaînes fonctionnent correctement pour les chaînes ASCII (on a remarqué que les caractères spéciaux "é" ne sont pas affichés correctement).
- La "*ConsoleDriver*" gère bien les caractères un par un.
- Les tampons de lecture et d'écriture sont bien implémentés et se termine par un `\0` afin d'éviter les débordements.
- Pas de bug détecté de notre part, on peut compiler et exécuter notre travail sans problème. On n'est pas sûrs d'avoir répondu correctement à tout ce qui nous a été demandé mais notre code ne génère pas de bug.

II. Points délicats

- Pour nous trois, la découverte du sujet n'a pas été simple. Le vocabulaire utilisé était totalement nouveau, donc comprendre ce qui nous était demandé était compliqué. Notamment, nous avons eu du mal à comprendre le passage entre le monde utilisateur et le monde noyau via `ReadMem` et `WriteMem`, ainsi que la gestion de la mémoire. La notion de Semaphore nous a également posé problème, mais nous avons réussi à la comprendre en cours.
- Une notion que nous avons mis un peu de temps à comprendre est la manipulation du code assembleur pour gérer la sortie de la console.
- La partie la plus difficile a été la partie bonus. L'implémentation de certaines méthodes comme `printf`. La principale difficultés était de comprendre le fichier `vsprintf.c`, d'adapter le code à notre projet avec l'implémentation des fonctions `isxdigit`, `isdigit`, `islower`, `toupper` et `strnlen`.
- La gestion des fuites mémoires nous faisait également peur mais l'outil `valgrind` nous a permis de gérer ces fuites.

Pour résoudre ces problème, notre enseignant en TD nous a forcément beaucoup aidé et nous avons aussi utilisé ChatGPT et des sites comme *Stackoverflow* ou même "*Koor*" lors de nos sessions à la maison, qui nous ont aidés.

III. Limitations

- Notre code n'est peut-être pas optimisé et reste simpliste, mais c'était un choix volontaire afin de rester dans une approche claire et fonctionnelle.
- Nous manquons peut-être de recul pour le critiquer de manière négative car nous avons suivi fidèlement le sujet, qui laissait peu de place à des erreurs majeures.
- Notre système Nachos ne gère que les caractères ASCII et ne prend pas en charge les caractères spéciaux et les caractères accentués s'affichent mal (é → Å©).

IV. Tests

Nous avons mis en place des fonctions de tests afin de vérifier nos implémentations. Nous avons testé des cas standards (chaînes valides) ainsi que des cas limites : chaînes vides, chaînes dépassant le buffer, ou contenant des caractères invalides. Cela nous a permis d'anticiper les situations problématiques et de valider le bon fonctionnement global.

Pour la partie Bonus, il fallait intégrer un `printf` en réutilisant le code `vsprintf.c` issu des sources de Linux donné par l'énoncé. La partie bonus avait plusieurs ajouts et modifications de fichiers dans Nachos :

Ajout du fichier `vsprintf.c` dans le répertoire `test/`, puis compilation en `vsprintf.o`.

Modification du `Makefile` : il fallait lier `vsprintf.o` aux programmes MIPS, tout en l'excluant de la liste des (PROGS).

Il fallait aussi ajouter les prototypes des fonctions `isxdigit`, `isdigit`, `islower`, `toupper` et `strnlen` dans le fichier `/test/vsprintf.c` et les implémentés.

Une fois ces fonctions définies, on pouvait compiler le fichier `vsprintf.c`. Le `printf` pouvait alors être construit en appelant `vsprintf` pour formater une chaîne puis en utilisant l'appel système `PutString` pour afficher le résultat dans le terminal.